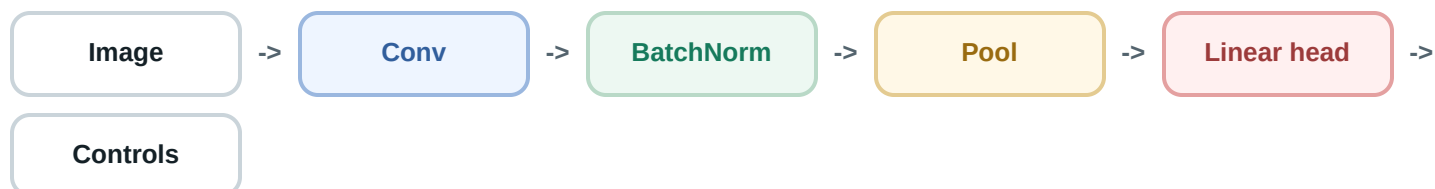


SidewalkPilot CNN Parameter Visual Guide

This explains why the older 1.x/2.x model has 672,877 parameters and why the Series 3 model has 5,533,010 parameters. The count comes from layer shapes, not from FP32 itself.

A parameter is one learnable number. FP32 only says each number uses 4 bytes. The architecture decides how many numbers exist.

The whole CNN at a glance



Conv layer

A convolution layer learns small image filters. Each filter slides across the image and looks for a pattern such as an edge, curb, shadow, or sidewalk texture.

```

Conv params =
out_channels x input_channels x
kernel_h x kernel_w
+ out_channels bias
  
```

BatchNorm layer

BatchNorm keeps channel values stable while training. Each channel gets two learnable numbers: scale and shift.

```

BatchNorm params =
2 x channels
  
```

Pooling

Pooling does not add learnable parameters here. It reshapes the feature map into a fixed size before the dense layers.

```

1.x/2.x pool = 64 x 4 x 8 = 2,048
features
3.x pool      = 160 x 6 x 10 = 9,600
features
  
```

Linear head

A linear layer is a big matrix of weights. This is where most of the parameters are.

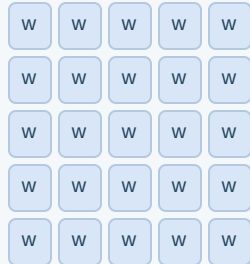
```

Linear params =
input_features x output_features
+ output_features bias
  
```

What a Conv filter is

The image has 3 color channels. A 5x5 conv filter has one 5x5 grid for each input channel. That means one filter has $3 \times 5 \times 5 = 75$ weights.

One 5x5 filter for one channel



25 weights for one channel.

Color image has 3 channels

B 5x5

G 5x5

R 5x5

3 channels $\times$ 25 weights = 75 weights per filter

Example: first 1.x/2.x conv layer

Input channels = 3

Output filters = 24

Kernel = 5×5

Params = $24 \times 3 \times 5 \times 5 + 24$ bias

Params = $24 \times 75 + 24$

Params = 1,824

Example: first 3.x conv layer

Input channels = 3

Output filters = 32

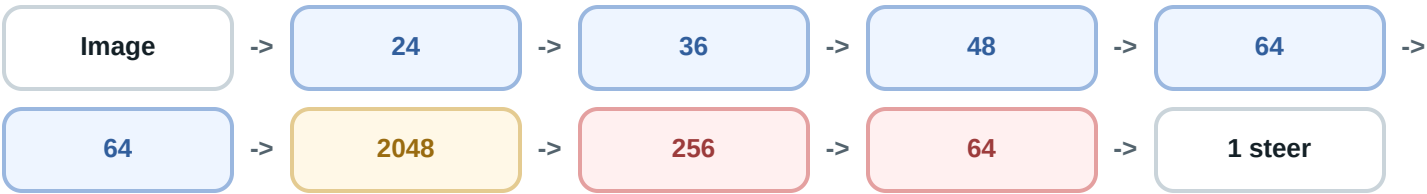
Kernel = 5×5

Params = $32 \times 3 \times 5 \times 5 + 32$ bias

Params = $32 \times 75 + 32$

Params = 2,432

1.x/2.x architecture: where 672,877 came from

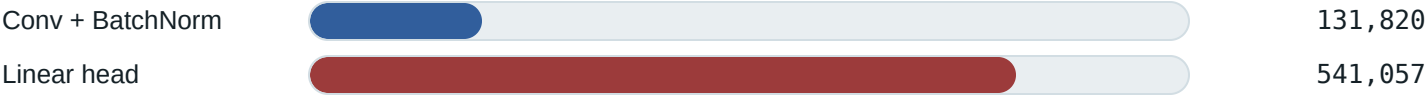


Part	Shape idea	Parameters
Convolutions	3->24->36->48->64->64	131,348
BatchNorm	2 values per channel	472
Linear head	2048->256->64->1	541,057
Total		672,877

The largest 1.x/2.x layer

Pool output = 64 x 4 x 8 = 2,048 features
First dense layer = 2,048 -> 256

Params = 2,048 x 256 + 256 bias
Params = 524,288 + 256
Params = 524,544



3.x architecture: where 5,533,010 came from



Part	Shape idea	Parameters
Convolutions	3->32->48->64->96->128->160	468,336
BatchNorm	2 values per channel	1,056
Linear head	9600->512->256->64->2	5,063,618
Total		5,533,010

The largest 3.x layer

Pool output = $160 \times 6 \times 10 = 9,600$ features

First dense layer = $9,600 \rightarrow 512$

Params = $9,600 \times 512 + 512$ bias

Params = $4,915,200 + 512$

Params = 4,915,712

Conv + BatchNorm		469,392
Linear head		5,063,618

Why the parameter count jumped by 8.22x

Old biggest layer

```
2,048 -> 256
2,048 x 256 + 256
= 524,544 params
```

New biggest layer

```
9,600 -> 512
9,600 x 512 + 512
= 4,915,712 params
```

The main multiplier

```
New biggest layer / old biggest layer
= 4,915,712 / 524,544
= 9.37x
```

Total model multiplier

```
Series 3 total / Series 1-2 total
= 5,533,010 / 672,877
= 8.22x
```

Change	Effect
Conv channels got wider	More filters can learn more visual patterns.
One extra conv block	The model can build deeper features before making a decision.
Pool changed from 64 x 4 x 8 to 160 x 6 x 10	The first dense layer receives 9,600 values instead of 2,048.
Dense head changed from 256 hidden units to 512 then 256	The decision part became much larger.
Output changed from 1 to 2	Steering plus throttle. This adds only a small number of parameters.

The biggest reason is not the second output. The biggest reason is that the first dense layer changed from 2,048 x 256 to 9,600 x 512.